

CS2900 Homework 2

Fawn Sannar 2026-09-24

Question 1

Source

```
#include <errno.h>
#include <inttypes.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdio.h>

// If any of this looks weird, please see my submission for Homework 1

static const uint8_t pc1[56] = {
    7, 15, 23, 31, 39, 47, 55, 63, 6, 14, 22, 30, 38, 46, 54, 62, 5,
    13, 21, 29, 37, 45, 53, 61, 4, 12, 20, 28, 1, 9, 17, 25, 33, 41,
    49, 57, 2, 10, 18, 26, 34, 42, 50, 58, 3, 11, 19, 27, 35, 43, 51,
    59, 36, 44, 52, 60
};

static const uint8_t pc2[48] = {
    42, 39, 45, 32, 55, 51, 53, 28, 41, 50, 35, 46, 33, 37, 44, 52, 30,
    48, 40, 49, 29, 36, 43, 54, 15, 4, 25, 19, 9, 1, 26, 16, 5, 11, 23,
    8, 12, 7, 17, 0, 22, 3, 10, 14, 6, 20, 27, 24
};

static const unsigned sched[16] = {
    1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1
};

static const uint8_t ip1[64] = {
    6, 14, 22, 30, 38, 46, 54, 62, 4, 12, 20, 28, 36, 44, 52, 60, 2,
    10, 18, 26, 34, 42, 50, 58, 0, 8, 16, 24, 32, 40, 48, 56, 7, 15,
    23, 31, 39, 47, 55, 63, 5, 13, 21, 29, 37, 45, 53, 61, 3, 11, 19,
    27, 35, 43, 51, 59, 1, 9, 17, 25, 33, 41, 49, 57
};

static const uint8_t ebit[48] = {
    0, 31, 30, 29, 28, 27, 28, 27, 26, 25, 24, 23, 24, 23, 22, 21, 20,
    19, 20, 19, 18, 17, 16, 15, 16, 15, 14, 13, 12, 11, 12, 11, 10, 9,
    8, 7, 8, 7, 6, 5, 4, 3, 4, 3, 2, 1, 0, 31
};

static const uint8_t s[8][64] = {
    {
        14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7,
        0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8,
        4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0,
        15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13
    },
    {
        15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10,
        3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5,
    },
    {
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
    },
    {
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
    },
    {
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
    },
    {
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
    },
    {
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
    },
    {
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
        1, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
        0, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0,
    },
}
```

```

    0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15,
    13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9,
},
{
    10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8,
    13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1,
    13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7,
    1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12,
},
{
    7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15,
    13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9,
    10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4,
    3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14,
},
{
    2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9,
    14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6,
    4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14,
    11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3,
},
{
    12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11,
    10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8,
    9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6,
    4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13,
},
{
    4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1,
    13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6,
    1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2,
    6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12,
},
{
    13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7,
    1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2,
    7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8,
    2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11,
},
};

static const uint8_t p[32] = {
    16, 25, 12, 11, 3, 20, 4, 15, 31, 17, 9, 6, 27, 14, 1, 22, 30, 24,
    8, 18, 0, 5, 29, 23, 13, 19, 2, 26, 10, 21, 28, 7
};

static const uint8_t ip2[64] = {
    24, 56, 16, 48, 8, 40, 0, 32, 25, 57, 17, 49, 9, 41, 1, 33, 26, 58,
    18, 50, 10, 42, 2, 34, 27, 59, 19, 51, 11, 43, 3, 35, 28, 60, 20,
    52, 12, 44, 4, 36, 29, 61, 21, 53, 13, 45, 5, 37, 30, 62, 22, 54,
    14, 46, 6, 38, 31, 63, 23, 55, 15, 47, 7, 39
};

uint64_t des_decrypt(uint64_t msg, uint64_t key) {
    // Permute initial key (PC-1)
    uint64_t pk = 0, k[16] = {};

```

```

for (int i = 0; i < 56; ++i) pk = pk << 1 | (1 & key >> pc1[i]);

// Create subkeys
uint64_t c = pk >> 28, d = pk & 0xffffffff;
for (int i = 0; i < 16; ++i) {
    // C and D are only ever used to create their corresponding
    // subkey, so there's absolutely no reason to store more than
    // one at a time
    c = (c << sched[i]) | (c >> (28 - sched[i]));
    d = (d << sched[i]) | (d >> (28 - sched[i]));
    // Permute concatenated subkey (PC-2)
    const uint64_t cd = (c << 28) | (d & 0xffffffff);
    for (int j = 0; j < 48; ++j) k[i] = k[i] << 1 | (1 & cd >> pc2[j]);
}

// Initial message permutation (IP-1)
uint64_t pmsg = 0;
for (int i = 0; i < 64; ++i) pmsg = pmsg << 1 | (1 & msg >> ip1[i]);
uint32_t l = pmsg >> 32, r = pmsg & 0xffffffff;

// Left-right dance
for (int i = 16 - 1; i >= 0; --i) { // BUT IN REVERSE!!!
    uint64_t rprev = r, e = 0;
    for (int j = 0; j < 48; ++j) e = e << 1 | (1 & r >> ebit[j]);
    e ^= k[i];

    uint32_t sb = 0;
    for (int j = 0; j < 8; ++j) {
        const uint8_t x = 0x3f & (e >> (6 * (7 - j))); // 6-bit block
        // Index into S-table using dimensional collapse via bithacks
        sb = sb << 4 | s[j][(15 & x >> 1) + ((1 & x) | (2 & (x >> 4))) * 16];
    }

    r = 0; // Store f directly into r
    for (int j = 0; j < 32; ++j) r = r << 1 | (1 & sb >> p[j]);
    r ^= l;
    l = rprev;
}

// Apply final permutation
pmsg = ((uint64_t)(r) << 32) | l;
for (int i = 0; i < 64; ++i) msg = msg << 1 | (1 & pmsg >> ip2[i]);
return msg;
}

int main(int argc, const char *argv[static argc]) {
    if (argc < 3) {
        fputs("Message and key required\n", stderr);
        fprintf(stderr, "Usage: %s MSG KEY\n", argv[0]);
        return 2;
    }

    bool badusage = false;
    char *endptr = nullptr;
    errno = 0;
    const uint64_t msg = strtoull(argv[1], &endptr, 16);

```

```

if (errno || endptr == argv[1]) {
    badusage = true;
    if (errno) perror("Bad msg"); else fputs("Bad msg\n", stderr);
}
errno = 0;
const uint64_t key = strtoull(argv[2], &endptr, 16);
if (errno || endptr == argv[2]) {
    badusage = true;
    if (errno) perror("Bad key"); else fputs("Bad key\n", stderr);
}
if (badusage) {
    fprintf(stderr, "Usage: %s MSG KEY\n", argv[0]);
    return 2;
}
printf("%016" PRIx64 "\n", des_decrypt(msg, key));
}

```

Output



```

~/Projects/cs2900/2: make 2.out
cc -std=c23 -O2 -Wall -Wextra -pedantic -o 2.out des_decrypt.c
~/Projects/cs2900/2: ./2.out 85E813540F0AB405 133457799BBCDFF1
0123456789abcdef
~/Projects/cs2900/2: ./2.out 1690dc44f583fd4e deadbeefdeadcode
fedcba9876543210
~/Projects/cs2900/2: 

```

Figure 1: Screenshot of DES decryption output

Question 2

Question 3

Question 4